

Project Report

Individual Assignment 2 - DApp Integration with Smart Contract

ALU Logo Registry & Ownership dApp

Abstract

This project delivers a complete decentralized application that reuses the ALU logo smart contracts from Formative 1 and makes them usable through a web interface. Users can connect a wallet, register a logo on-chain using its SHA-256 fingerprint, verify authenticity without a server, and inspect or distribute ALUT ownership shares from the browser.

Introduction

The smart contracts built earlier solved the blockchain side of the problem but not the usability side. Without a frontend, only someone comfortable with a terminal and Hardhat commands could interact with the logo record or token supply. The dApp fills that gap by becoming the front door to the contracts.

The result is a more realistic Web3 product: the smart contracts remain the backend source of truth, the wallet provides identity and transaction signing, and the browser frontend makes the workflow understandable for non-technical users.

Architecture Overview

The application is made of a plain JavaScript frontend, an injected browser wallet, a local Hardhat blockchain, and two deployed Solidity contracts. The frontend uses ethers.js and the contracts' ABI definitions to create contract instances for both read-only calls and signed transactions.

- ALUAssetRegistry stores the unique ERC-721 record for the logo and its SHA-256 hash.
- ALULogoToken stores the ERC-20 ownership shares with a fixed supply of 1,000,000 ALUT.
- The Verify page uses read-only calls, so it works even without a wallet connection.
- The Register and Distribution flows require a signer because they change blockchain state.

Feature Walkthrough

Home Page

The landing page introduces the three main journeys in the dApp: register a logo, verify a logo, and inspect or distribute ownership shares. It also shows wallet status and ALUT balance in the header.

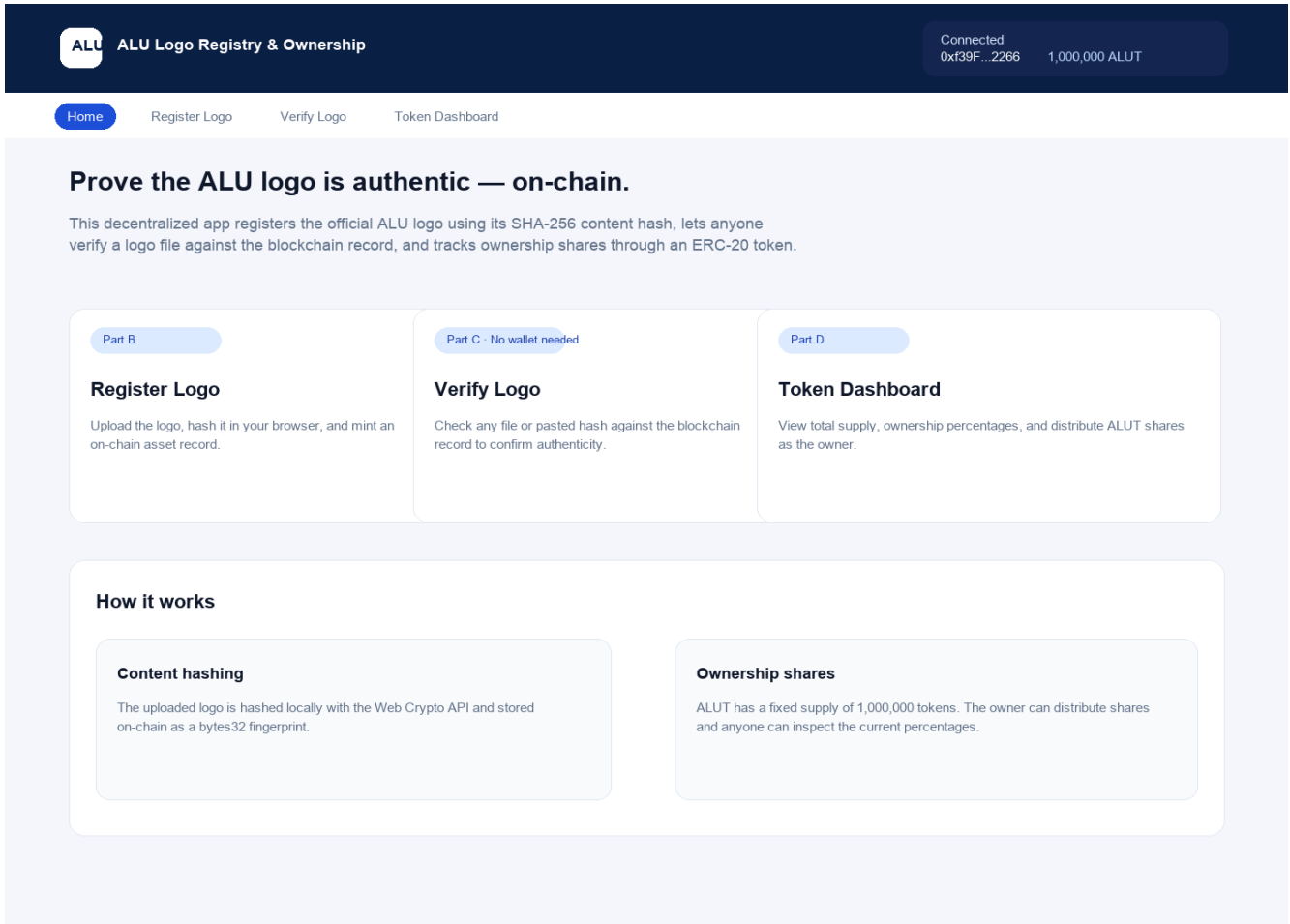


Figure 1. Home page of the ALU Logo Registry & Ownership dApp.

Register Logo Page

The Register page reads the uploaded file locally, computes its SHA-256 digest in the browser, previews the selected logo, and pre-fills the bytes32 hash into the registration flow. After the wallet signs the transaction, the page calls `registerAsset()` and shows the minted token ID.

Generating the hash in the browser is important because the file itself never needs to be uploaded to a server. If a duplicate hash is submitted, the smart contract rejects it and the frontend surfaces the error.

ALU

ALU Logo Registry & Ownership

Connected
0xf39F...22661,000,000 ALUT

[Home](#)[Register Logo](#)[Verify Logo](#)[Token Dashboard](#)

Register Logo

Upload the ALU logo file, compute its SHA-256 hash in the browser, and send a wallet-approved registerAsset() transaction.

1. Hash the logo file

alu-logo.png



SHA-256 content hash (bytes32)

0x727b0a610a5e22c083efc8b467c1580cad4b2626d8a6171d686a99d6a220a560

2. Register on-chain

Asset name

ALU Official Logo

File type

png

Content hash to register

0x727b0a61...a220a560

Register Logo

Success: token ID 2 minted after wallet approval.

Figure 2. Register page showing local hashing and the on-chain registration form.

Verify Logo Page

The Verify page supports both file upload and pasted hash verification. It computes the file digest locally when needed, calls `verifyLogoIntegrity()` with the known token ID, and presents a clear green authentic verdict or red modified warning.

No wallet is required for this page because `verifyLogoIntegrity()` and `getAsset()` are read-only calls. They read existing blockchain data, do not change state, and therefore cost no gas.

Page 3

ALU

ALU Logo Registry & Ownership

Connected
0xf39F...22661,000,000 ALUT

[Home](#)[Register Logo](#)[Verify Logo](#)[Token Dashboard](#)

Verify Logo

Verify any logo file or a pasted hash against the official on-chain fingerprint. This page works without a wallet because it only performs read-only calls.

Option 1 — Upload a file

alu-logo.png

0x727b0a610a5e22c083efc8b467c1580cad4b2626d8a6171d686a99d6a220a560

Option 2 — Paste a hash

0x727b0a610a5e22c083efc8b467c1580cad4b2626d8a6171d686a99d6a220a560

1

Verify Against Blockchain

☒ Logo Verified: This is the authentic ALU logo.

On-chain record

Asset name: ALU Official Logo
File type: png

Registered by: 0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266
Registered at: 2026-07-08 16:35

Figure 3. Verify page showing a successful authentic-logo result and the on-chain metadata card.

Token Dashboard Page

The dashboard reads the total supply, current wallet balance, ownership percentage, and contract owner directly from ALULogoToken. When the owner wallet is connected, the page also enables the distributeShares() form so ownership can be transferred to another wallet.

The onlyOwner modifier protects distributeShares() at the contract level. If a non-owner wallet attempts the call, the transaction reverts and balances remain unchanged.

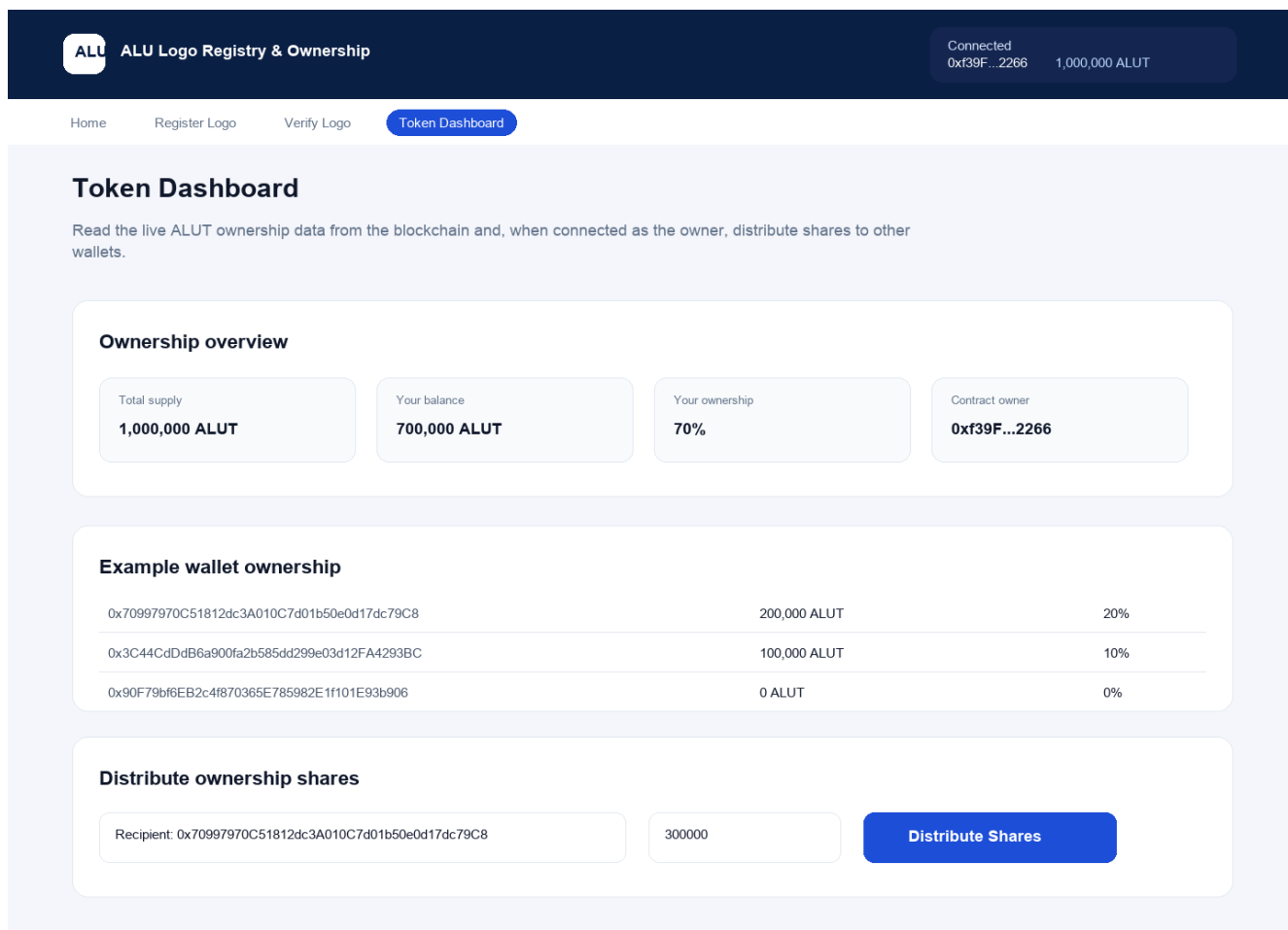


Figure 4. Token dashboard showing ownership statistics and the owner-only share distribution form.

Wallet and Web3 Integration

The frontend uses ethers.js to talk to both contracts. The ABI is necessary because it tells the frontend which functions exist, how to encode inputs, and how to decode outputs. Without the ABI, the browser would have a contract address but no reliable way to interact with it.

Read-only calls such as `totalSupply()`, `balanceOf()`, `verifyLogoIntegrity()`, `getAsset()`, `owner()`, and `ownershipPercentage()` do not require gas or signatures. Transactions such as `registerAsset()` and `distributeShares()` require the user to sign in their wallet because only the wallet should control the private key.

Verification Page

The verification feature uses the Web Crypto API to compute SHA-256 over the raw file bytes directly in the browser. The digest is converted to a 0x-prefixed bytes32 string and compared to the hash stored on-chain. Because no backend is needed, the design is simple, private, and fully serverless.

A realistic user of this page could be a design team, a student club, or an external printing vendor that needs to confirm the logo file they received is the authentic ALU version before publishing it.

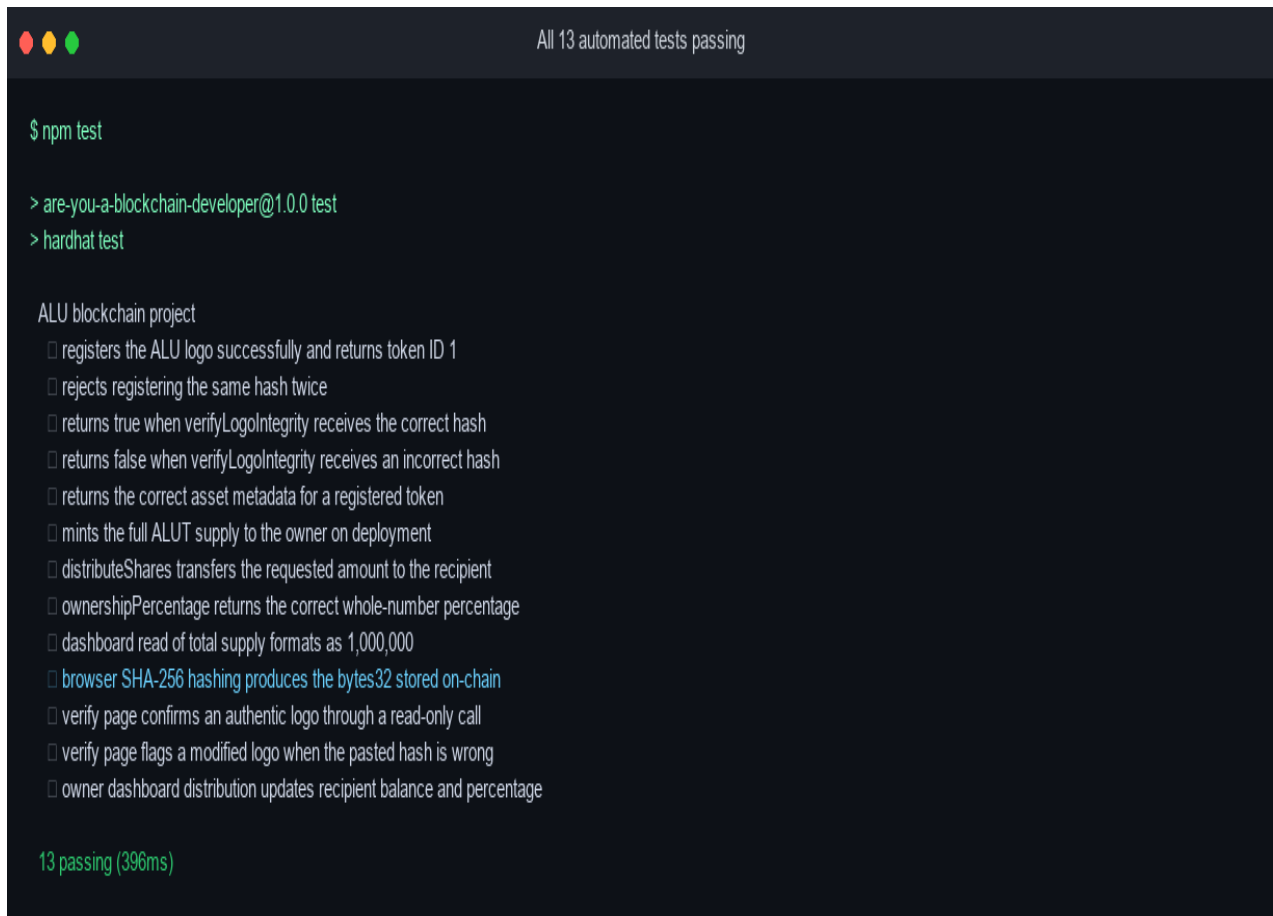
Token Dashboard

The dashboard translates the ERC-20 state into plain-language ownership information. Someone at ALU, such as the brand office or administration, could receive ALUT shares to represent a stake in the tokenised ownership model used by the project.

If ALU wanted token holders to vote on decisions proportional to their holdings, the contract would need governance additions such as proposal creation, vote recording, quorum requirements, and token-weighted result calculation.

Test Results

The final repository includes all thirteen required tests: the original eight contract tests plus five additional frontend-aligned integration tests. The added tests confirm the dashboard reads the 1,000,000 total supply correctly, browser-style SHA-256 hashing returns a valid bytes32 hash, the verify flow succeeds for the real hash and fails for an incorrect one, and share distribution updates balances and percentages.

A terminal window with a dark background. At the top, there are three colored circles (red, yellow, green) and the text "All 13 automated tests passing". Below this, the command "\$ npm test" is entered. The output shows the command "> are-you-a-blockchain-developer@1.0.0 test" and "> hardhat test". A section titled "ALU blockchain project" lists 13 tests, each preceded by a checkbox. All checkboxes are checked. The tests are: registers the ALU logo successfully and returns token ID 1, rejects registering the same hash twice, returns true when verifyLogoIntegrity receives the correct hash, returns false when verifyLogoIntegrity receives an incorrect hash, returns the correct asset metadata for a registered token, mints the full ALUT supply to the owner on deployment, distributeShares transfers the requested amount to the recipient, ownershipPercentage returns the correct whole-number percentage, dashboard read of total supply formats as 1,000,000, browser SHA-256 hashing produces the bytes32 stored on-chain, verify page confirms an authentic logo through a read-only call, verify page flags a modified logo when the pasted hash is wrong, and owner dashboard distribution updates recipient balance and percentage. At the bottom, it says "13 passing (396ms)".

```
$ npm test

> are-you-a-blockchain-developer@1.0.0 test
> hardhat test

ALU blockchain project
  ☒ registers the ALU logo successfully and returns token ID 1
  ☒ rejects registering the same hash twice
  ☒ returns true when verifyLogoIntegrity receives the correct hash
  ☒ returns false when verifyLogoIntegrity receives an incorrect hash
  ☒ returns the correct asset metadata for a registered token
  ☒ mints the full ALUT supply to the owner on deployment
  ☒ distributeShares transfers the requested amount to the recipient
  ☒ ownershipPercentage returns the correct whole-number percentage
  ☒ dashboard read of total supply formats as 1,000,000
  ☒ browser SHA-256 hashing produces the bytes32 stored on-chain
  ☒ verify page confirms an authentic logo through a read-only call
  ☒ verify page flags a modified logo when the pasted hash is wrong
  ☒ owner dashboard distribution updates recipient balance and percentage

13 passing (396ms)
```

Figure 5. All thirteen automated tests passing.

Supporting Evidence

The following terminal evidence captures the setup steps used by the dApp.

```
SHA-256 of alu-logo.png

$ npm run hash:logo

> are-you-a-blockchain-developer@1.0.0 hash:logo
> node scripts/hash-logo.js

SHA-256: 727b0a610a5e22c083efc8b467c1580cad4b2626d8a6171d686a99d6a220a560
bytes32: 0x727b0a610a5e22c083efc8b467c1580cad4b2626d8a6171d686a99d6a220a560
```

Figure 6. SHA-256 hash generation for alu-logo.png.

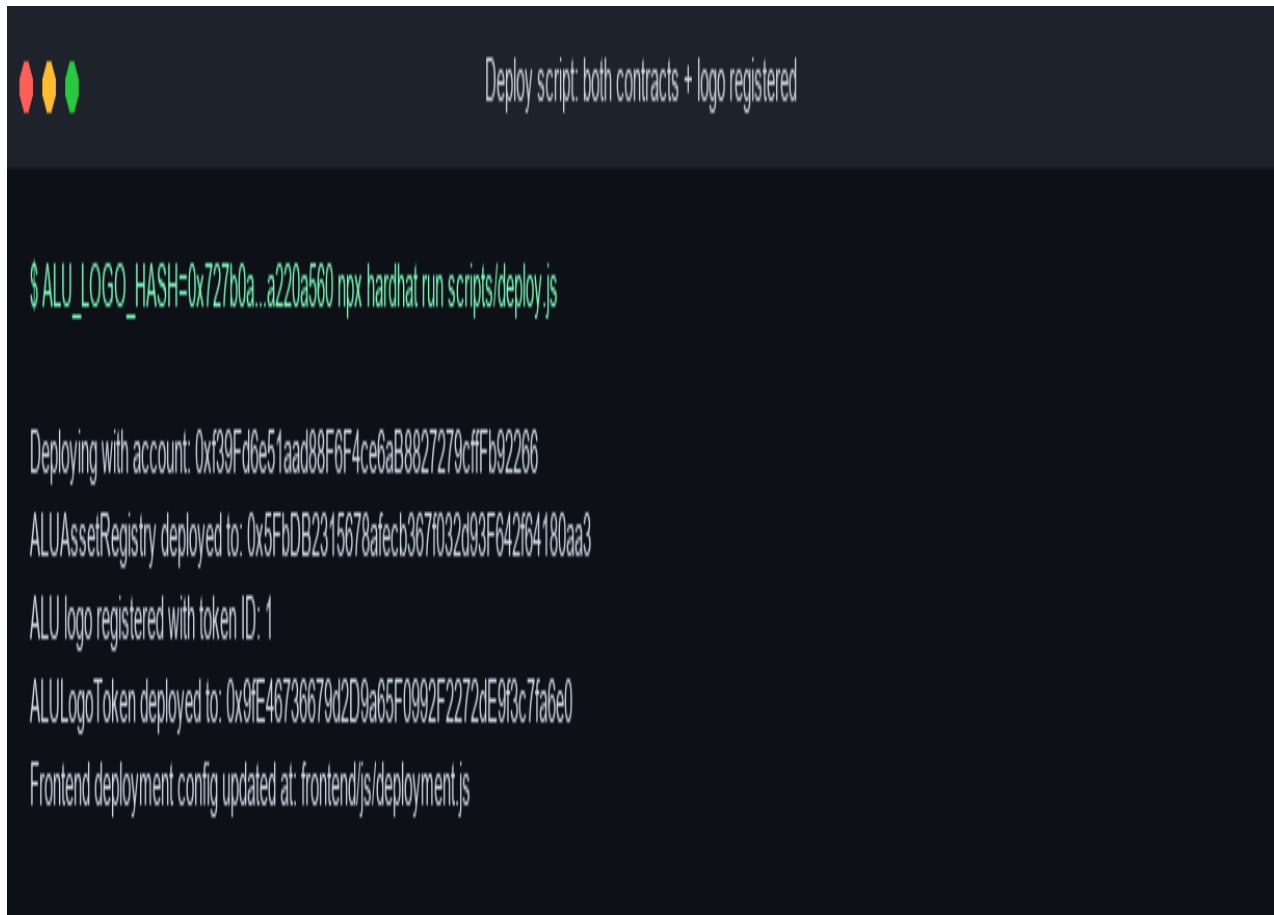
```
Local Hardhat node with test wallet addresses

$ npx hardhat node
Started HTTP and WebSocket JSON-RPC server at http://127.0.0.1:8545/

Accounts
=====

WARNING: These accounts, and their private keys, are publicly known.
Account #0: 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb932266 (10000 ETH)
Account #1: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8 (10000 ETH)
Account #2: 0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC (10000 ETH)
Account #3: 0x90F79bf6EB2c4f870365E785982E1f101E93b906 (10000 ETH)
```

Figure 7. Local Hardhat node with pre-funded test wallets.



```
Deploy script: both contracts + logo registered

$ ALU_LOGO_HASH=0x727b0a...a220a560 npx hardhat run scripts/deploy.js

Deploying with account: 0xf39Fd6e51aad88F6F4ce6aB8827279cFfFb92266
ALUAssetRegistry deployed to: 0x5FbDB2315678afecb367f032d93F642f64180aa3
ALU logo registered with token ID: 1
ALULogoToken deployed to: 0x9fE46736679d2D9a65F0992F2272dE9f3c7fa6e0
Frontend deployment config updated at: frontend/js/deployment.js
```

Figure 8. Deployment of both contracts and registration of the official ALU logo.

Conclusion

This assignment demonstrated that a smart contract becomes much more useful when it is paired with a thoughtful frontend. The biggest lesson was how browser logic, wallet signatures, and on-chain state each play a different role in a complete Web3 product.

With more time, the next improvements would be deploying to a public testnet, adding broader wallet support, strengthening validation, and extending ALUT into a governance-enabled token system.

GitHub Link

<https://github.com/tejirijesse/are-you-a-blockchain-developer>